From Linear Models to Neural Networks

Linear & logistic regression, neurons, MLPs, universal approximation

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Recall from Lecture 1

Every loss was a negative log-likelihood, $\ell = -\log p_\theta(y \mid x)$:

MSE $\leftrightarrow$ **Gaussian** likelihood

cross-entropy $\leftrightarrow$ **Bernoulli / Categorical** likelihood

Today: $p_\theta(y \mid x)$ is parameterized by a neural network.

The key transition

Old ML models — one affine map:

$$f_\theta(x) = w^\top x + b$$

Neural networks — many affine maps + nonlinearities:

$$f_\theta(x) = W_L\, \varphi(W_{L-1}\varphi(\ldots\varphi(W_1 x + b_1))) + b_L$$

Main question. What changes when we replace **one** affine map by **many** affine maps separated by nonlinearities?

Linear regression

For $x \in \mathbb{R}^d$:

$$\hat{y} = w^\top x + b$$

Absorb the bias by appending a 1 to the input:

$$\hat{y} = \theta^\top \tilde{x}, \qquad \tilde{x} = \begin{pmatrix} 1 \\ x \end{pmatrix}$$

One weight vector, one hyperplane. This is the whole model.

Assume Gaussian noise around the line:

$$Y \mid x, w, b \sim \mathcal{N}(w^\top x + b, \sigma^2)$$

$$-\log p(y \mid x, w, b) = \frac{(y - w^\top x - b)^2}{2\sigma^2} + C$$

linear regression + MSE = Gaussian MLE

Matrix form

Dataset $X \in \mathbb{R}^{n \times d}$, targets $y \in \mathbb{R}^n$:

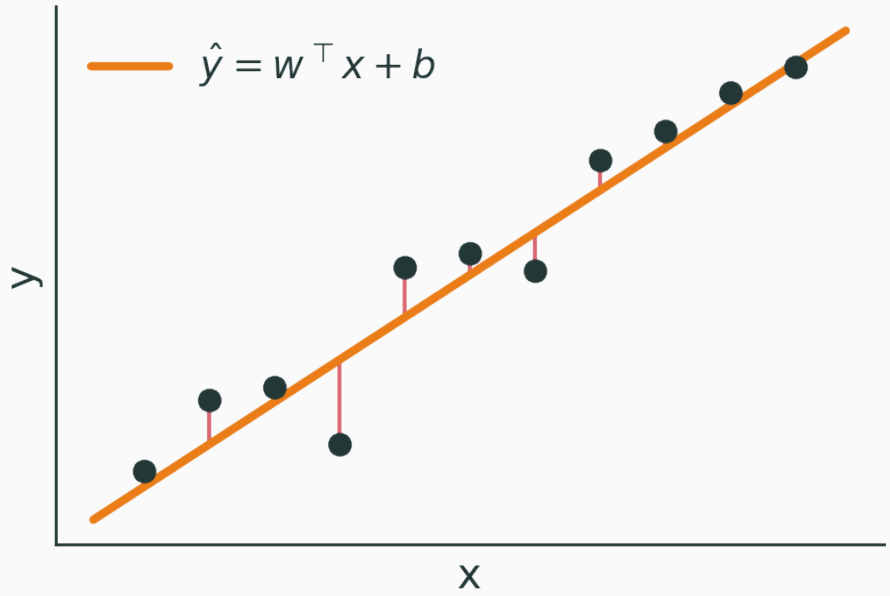
$$\hat{y} = Xw + b\mathbf{1} \quad \Longrightarrow \quad \hat{y} = \tilde{X}\theta$$

Loss over the whole dataset:

$$\mathcal{L}(\theta) = \frac{1}{n}\|y - \tilde{X}\theta\|_2^2$$

Geometry of linear regression

V



Linear regression fits a **hyperplane**: a line for $d = 1$, a plane for $d = 2$, a hyperplane in general. Residuals are the vertical gaps.

Closed-form solution

optional

If $\tilde{X}^\top \tilde{X}$ is invertible:

$$\hat{\theta} = \left(\tilde{X}^\top \tilde{X}\right)^{-1} \tilde{X}^\top y$$

But deep learning uses **gradient-based** optimization instead. Why?

- huge parameter count · nonlinear models · minibatches · streaming data

Gradient descent preview

For the MSE loss $\mathcal{L}(\theta) = \frac{1}{n}\left\|y - \tilde{X}\theta\right\|_2^2$:

$$\nabla_\theta \mathcal{L} = -\frac{2}{n}\,\tilde{X}^\top\left(y - \tilde{X}\theta\right)$$

$$\theta \leftarrow \theta - \eta\,\nabla_\theta \mathcal{L}$$

This update **is** what backpropagation will compute — for far more complex f_θ.

Interactive: linear regression & gradient descent

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

Fit a line by gradient descent — watch the **loss surface** and the **parameter trajectory** as you change the learning rate. Controls: slope w · intercept b · noise · learning rate · GD steps.

Q. What happens when the learning rate is too large?

Logistic regression

A linear score is not a probability

The linear score is unbounded:

$$z = w^\top x + b \in \mathbb{R}$$

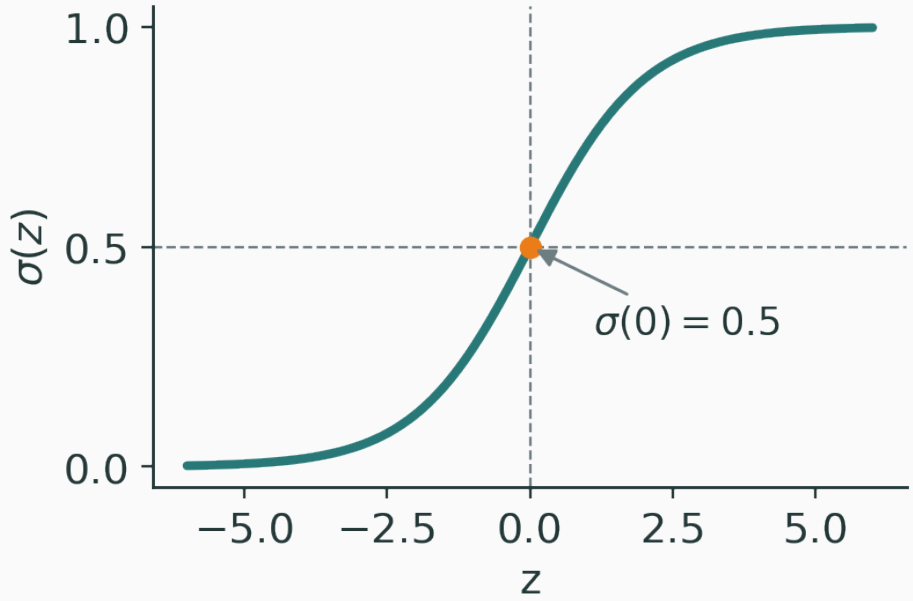
But a binary label needs

$$p(y = 1 \mid x) \in [0, 1].$$

squash it: $p(y = 1 \mid x) = \sigma(z)$

The sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Properties:

- $\sigma(z) \in (0, 1)$
- $\sigma(0) = 0.5$
- $\sigma(-z) = 1 - \sigma(z)$

The logistic regression model

$$z = w^\top x + b$$

$$p_\theta(y = 1 \mid x) = \sigma(z), \qquad p_\theta(y = 0 \mid x) = 1 - \sigma(z)$$

$$Y \mid x \sim \text{Bernoulli}(\sigma(w^\top x + b))$$

Binary cross-entropy loss

D

For $y \in \{0, 1\}$ with $\hat{p} = \sigma(w^\top x + b)$:

$$\mathcal{L} = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$$

This is exactly the Bernoulli NLL of the compact form

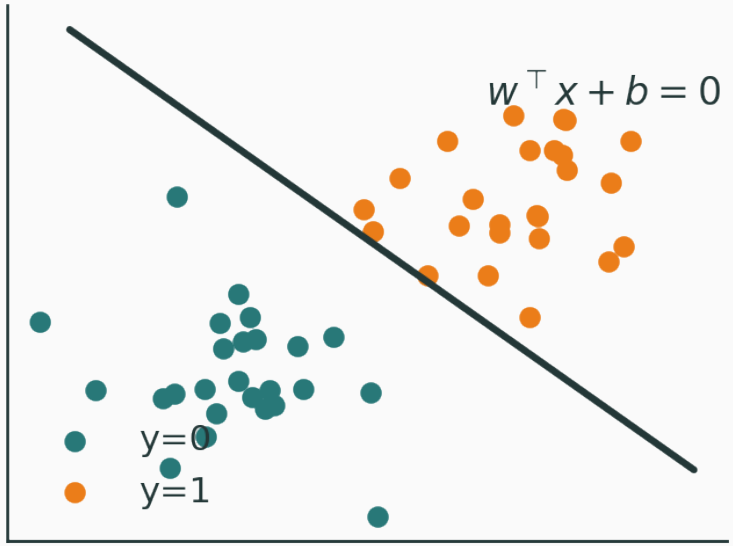
$$p(y \mid x) = \hat{p}^y (1 - \hat{p})^{1-y}.$$

Decision boundary

$$\hat{y} = \begin{cases} 1 \ \sigma(w^\top x + b) \geq 0.5 \\ 0 \ \text{otherwise} \end{cases}$$

Since $\sigma(z) \geq 0.5 \iff z \geq 0$, the boundary is

$$w^\top x + b = 0.$$



Logistic regression is still a linear classifier

The **output probability** is nonlinear in z (the sigmoid).

The **decision boundary** is linear in x: $w^\top x + b = 0$.

logistic regression = linear classifier + sigmoid

With $\hat{p} = \sigma(w^\top x + b)$ and BCE loss, the algebra collapses:

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{p} - y$$

$$\nabla_w \mathcal{L} = (\hat{p} - y)\, x$$

Same “predicted – target” form as Lecture 1′s softmax gradient. Keep an eye on it.

Interactive: logistic regression

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/softmax-temperature

Move a decision line over 2-D points and watch the **probability heatmap**, the **boundary**, and the **BCE loss** respond. Controls: line angle · bias · sigmoid steepness · threshold · add/remove points.

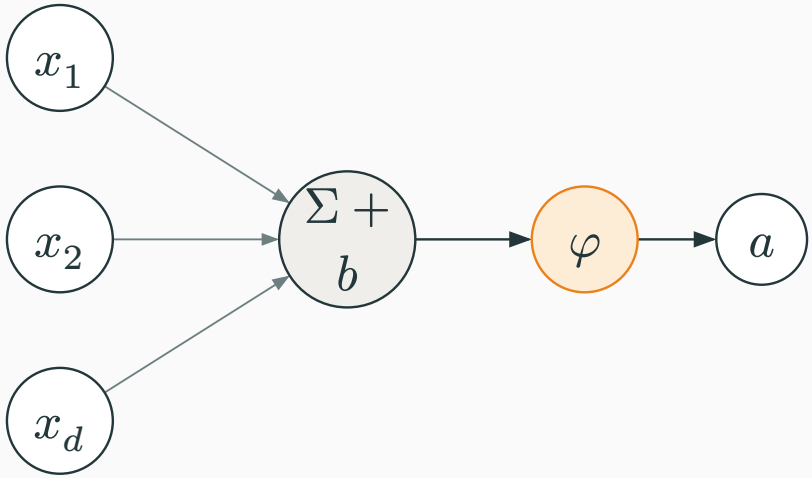
Q. How does the decision boundary move when b changes?

From logistic regression to a neuron

A neuron

A neuron applies an affine map, then an activation:

$$z = w^\top x + b, \qquad a = \varphi(z)$$



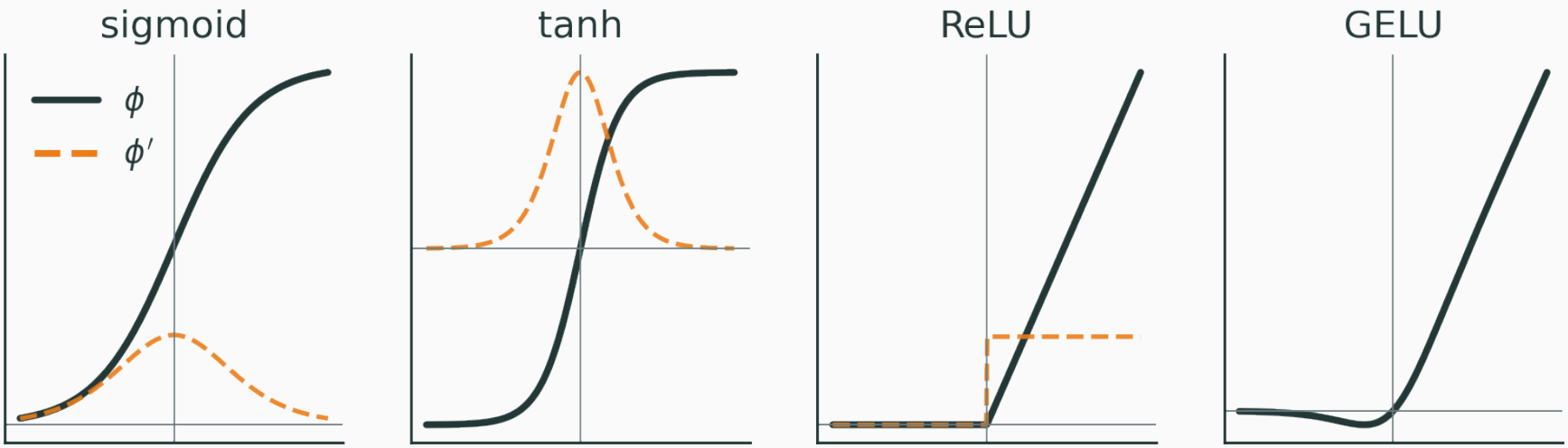
Logistic regression is one neuron

Take the activation to be the sigmoid:

$$a = \sigma(w^\top x + b) = p(y = 1 \mid x)$$

logistic regression is the simplest neural network — a single sigmoid neuron

Common activation functions



sigmoid → gates/probabilities · tanh → zero-centred · **ReLU** → default hidden unit · GELU → Transformers

Why we need a nonlinearity

D

Stack two **linear** layers:

$$h_1 = W_1 x + b_1, \quad h_2 = W_2 h_1 + b_2$$

$$h_2 = \underbrace{W_2 W_1}_{\text{one matrix}} x + \underbrace{(W_2 b_1 + b_2)}_{\text{one vector}}$$

Without a nonlinear activation, **depth collapses to a single linear layer.**

Interactive: activation functions

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Compare activations and their **derivatives**, and see where gradients vanish.
Controls: activation type · input z · show derivative · compare saturation.

Q. Where does the sigmoid saturate, and why might that hurt training?

Multi-class classification

From binary to multi-class

Binary — one score:

$$z = w^\top x + b$$

Multi-class — one score per class:

$$z_k = w_k^\top x + b_k, k = 1, ..., K$$

Each class gets its own logit; stack them into $z = Wx + b$.

Softmax regression

$$p(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \quad z = Wx + b$$

Also called **multinomial logistic regression**. Generalizes the sigmoid to K classes.

Multi-class cross-entropy

D

For a one-hot target y:

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log p_k$$

Only the true class survives:

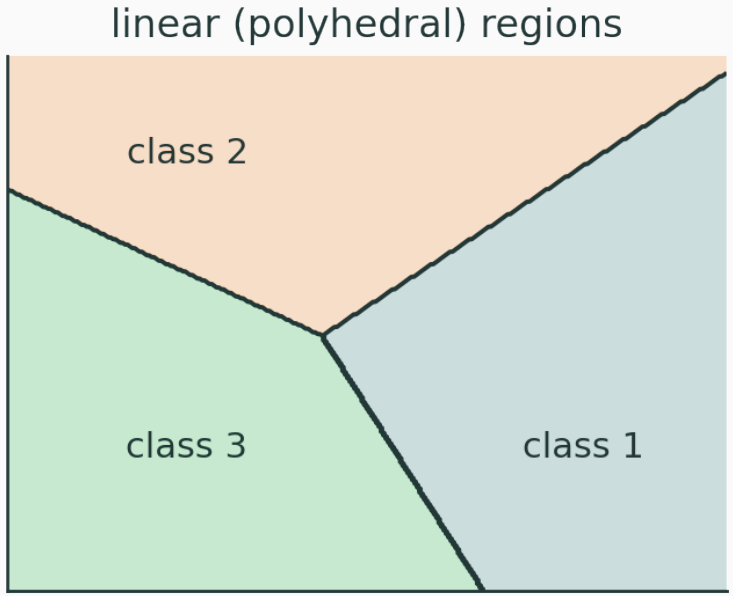
$$\mathcal{L} = -\log p_{\text{true class}}$$

Geometry of softmax regression

V

Boundary between classes i, j occurs when $z_i = z_j$:

$$(w_i - w_j)^\top x + (b_i - b_j) = 0$$



straight edges → linear / polyhedral regions

The limitation of softmax regression

Softmax regression only draws **linear** boundaries. It fails on:

- XOR
- concentric circles
- spirals
- real image / audio / text patterns

Interactive: softmax classifier I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/softmax-temperature

Three class weight vectors, a movable point, and a temperature T — watch the logits, the softmax probabilities, and the **linear decision regions**.

Q. Why are the decision regions straight-edged?

The multilayer perceptron

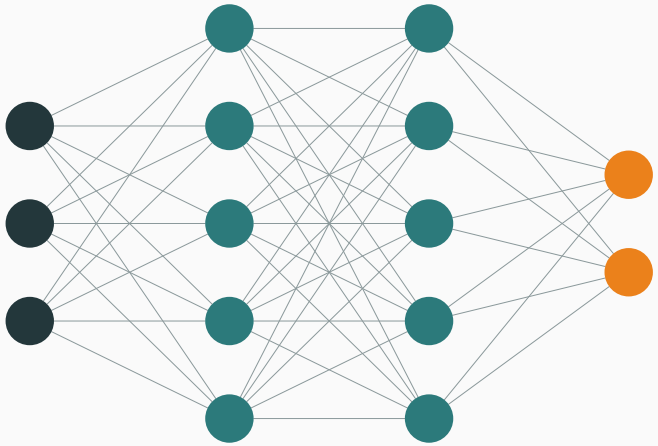
Add a hidden layer

Instead of $z = Wx + b$, insert a nonlinear layer first:

$$h = \varphi(W_1 x + b_1), \qquad z = W_2 h + b_2$$

For classification, $p = \mathrm{softmax}(z)$. This is a **one-hidden-layer MLP**.

MLP architecture



$$x \in \mathbb{R}^d$$
$$W_1 \in \mathbb{R}^{m \times d}$$
$$h \in \mathbb{R}^m$$
$$W_2 \in \mathbb{R}^{K \times m}$$
$$z \in \mathbb{R}^K$$

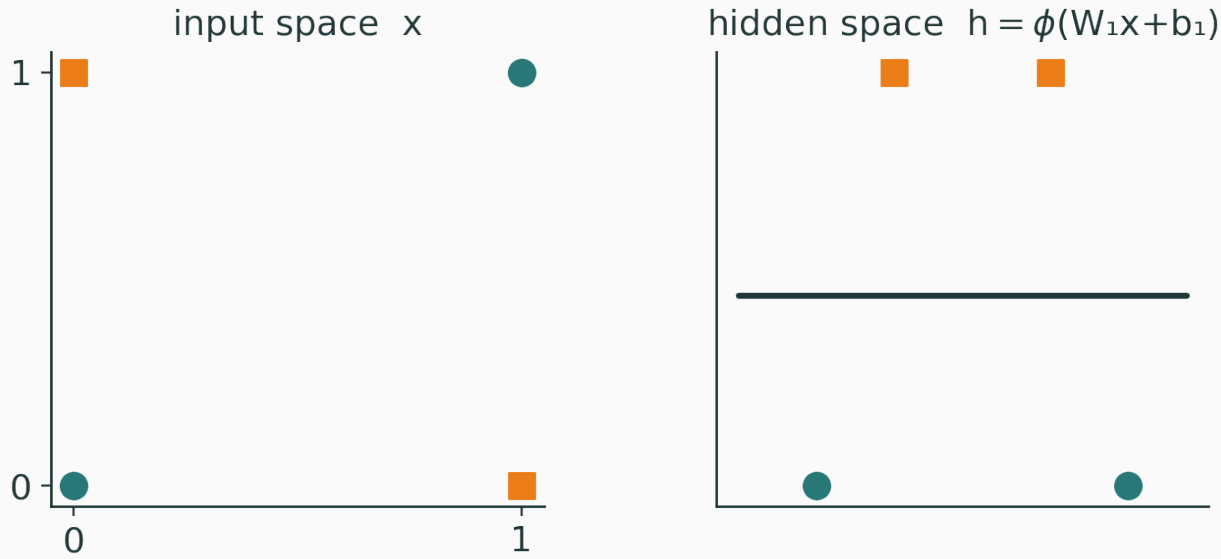
Each hidden neuron creates a feature

$$h_j = \varphi\left(w_j^\top x + b_j\right)$$

- the first layer **learns features**;
- the output layer **combines** them.

For ReLU, $h_j = \max\left(0, w_j^\top x + b_j\right)$ — each unit is active on one side of a hyperplane.

The feature-transformation view



an MLP learns a representation where classification becomes linear

MLP for binary vs multi-class classification

Binary

$$h = \varphi(W_1 x + b_1)$$

$$z = w_2^\top h + b_2$$

$$p = \sigma(z), \quad \mathcal{L} = \mathrm{BCE}$$

Multi-class

$$h = \varphi(W_1 x + b_1)$$

$$z = W_2 h + b_2$$

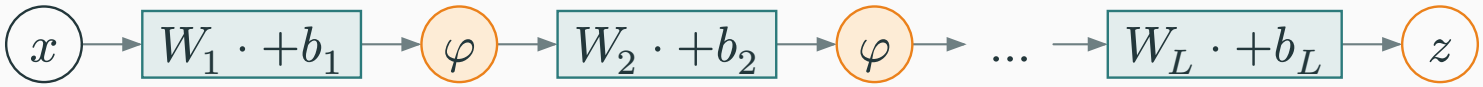
$$p = \mathrm{softmax}(z), \quad \mathcal{L} = \mathrm{CE}$$

Same output layer as before — only the **representation** h changed.

Multiple hidden layers

$$h^{(1)} = \varphi(W_1 x + b_1), \quad h^{(2)} = \varphi\big(W_2 h^{(1)} + b_2\big), \quad \ldots$$

$$z = W_L h^{(L-1)} + b_L$$



affine $\rightarrow$ nonlinearity $\rightarrow$ affine, stacked L deep

Then $p = \mathrm{softmax}(z)$ for classification, or $\hat{y} = z$ for regression.

Depth vs width

Width — more neurons per layer → more features at one level.

Depth — more layers → **compose** features into a hierarchy.

pixels → edges → motifs → objects

characters → words → phrases → meaning

Interactive: MLP decision boundaries

I

INTERACTIVE

(to build in interactive-articles) — train a small MLP on **linear / XOR / circles / spirals** and watch the boundary form. Controls: hidden units · layers · activation · learning rate · steps.

Q. What changes when you add hidden **units** vs when you add **layers**?

Why MLPs represent complex functions

ReLU networks are piecewise linear

V

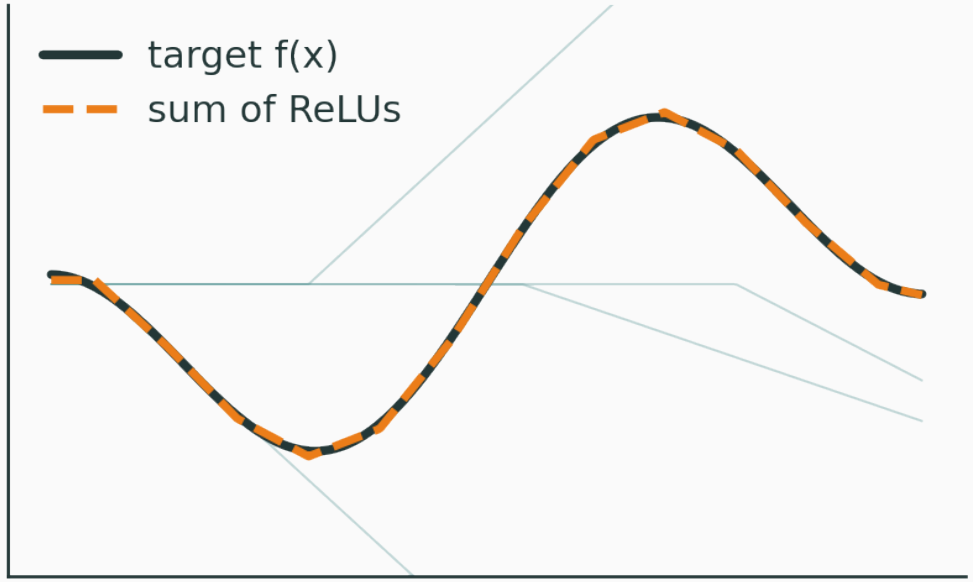
$$\varphi(z) = \max(0, z)$$

- one ReLU → a hinge;
- a sum of ReLUs → a piecewise-linear curve;
- many ReLUs → arbitrarily fine approximation.

Building functions from ReLUs

A one-hidden-layer scalar network:

$$f(x) = \sum_{j=1}^{m} a_j \, \mathrm{ReLU}\left(w_j x + b_j\right) + c$$



each hidden unit adds one kink

The universal approximation theorem

D

Let f be continuous on a compact domain. A feedforward network with **one** hidden layer and a suitable nonlinearity can approximate f arbitrarily well:

$$\forall\, \varepsilon > 0, \exists\, \text{MLP } g \quad \text{s.t.} \quad \sup_x |f(x) - g(x)| < \varepsilon$$

What the theorem does **not** say

optional

Universal approximation does **not** promise:

- that training will **find** that function;
- that **finite data** is enough;
- that a **small** network suffices;
- that it will **generalize**.

expressivity $\neq$ trainability $\neq$ generalization

Why depth still matters

Even though one hidden layer is universal, depth can be **exponentially** more efficient — and composition is natural:

$$f(x) = f_L \circ f_{L-1} \circ \ldots \circ f_1(x)$$

Images, language, and program-like computation are all **hierarchical** — depth mirrors that structure.

Interactive: function approximation I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/universal-approximation

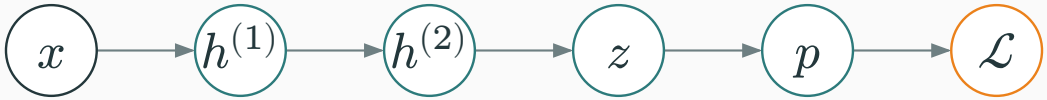
Nielsen's visual proof, by hand: crank a sigmoid into a step, glue steps into bumps, stack bumps to **design any 1-D curve**, then tile towers across a 3-D surface.

Q. Does more expressivity always make optimization easier?

Connecting to deep-learning practice

One example through an MLP

Forward pass, then a single scalar loss:



For multi-class: $\mathcal{L} = -\log p_y$. Next lecture reverses these arrows.

Parameters of an MLP

D

Layer ℓ: $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, $b_\ell \in \mathbb{R}^{d_\ell}$. Total:

$$\sum_{\ell=1}^{L} (d_\ell \, d_{\ell-1} + d_\ell)$$

Exercise. Input 784, hidden 128, output 10:

$$784 \times 128 + 128 + 128 \times 10 + 10 = 101{,}770.$$

The standard supervised DL template

Regression

$$z = f_\theta(x)$$

$$\mathcal{L} = (y - z)^2$$

Binary

$$z = f_\theta(x), p = \sigma(z)$$

$$\mathcal{L} = \mathrm{BCE}$$

Multi-class: $z = f_\theta(x), p = \mathrm{softmax}(z), \mathcal{L} = \mathrm{CE}.$

pick the output + likelihood; the loss follows

Summary

V

Model	Output	Distribution	Loss	Boundary
Linear regression	$w^\top x + b$	Gaussian	MSE	–
Logistic regression	$\sigma(w^\top x + b)$	Bernoulli	BCE	linear
Softmax regression	$\mathrm{softmax}(Wx + b)$	Categorical	CE	linear
MLP classifier	$\mathrm{softmax}(f_\theta(x))$	Categorical	CE	**nonlinear**

Final mental model — one idea

linear regression = linear score + Gaussian likelihood

logistic regression = linear score + sigmoid + Bernoulli

softmax regression = linear scores + softmax + categorical

MLP = learned nonlinear features + the same output likelihood

We now have $x \to f_\theta(x) \to \mathcal{L}$.

Next: how to get $\frac{\partial \mathcal{L}}{\partial \theta}$

computation graphs · the chain rule · backpropagation · automatic differentiation